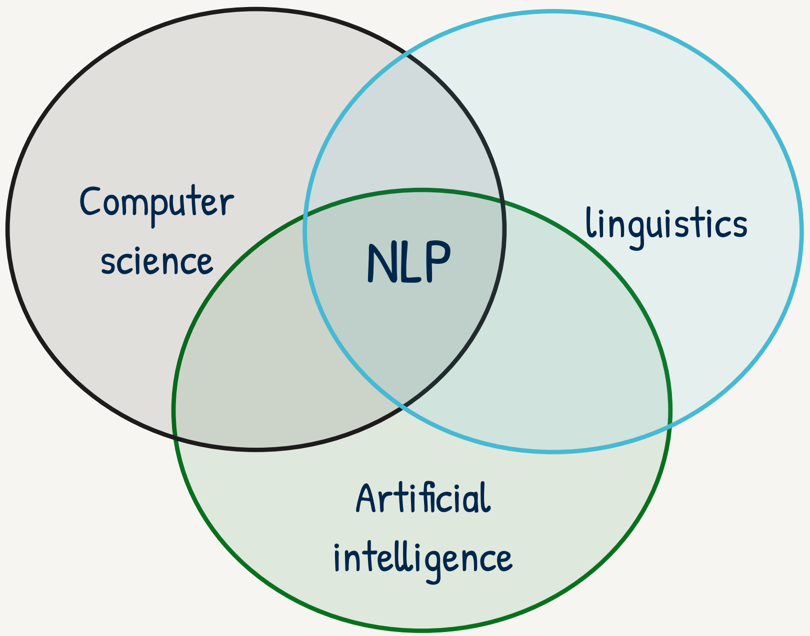
أكاديمية كاوست
KAUST ACADEMY

Day 1: Introduction to NLP
Summary

Contributed by: Hassan Mohammed Nasr

- Natural Language Processing (NLP) teaches machines to understand, interpret, and generate human language.



- Goal: Enable machines to understand and generate human language, as well as to develop systems that can process and analyze large amounts of text data.

Examples:

- Chatbots
- Machine translation
- Sentiment Analysis
- Information extraction and retrieval

NLP Pipeline

1 – Feature Extractor

- Cleaning and preparing raw text for analysis by lowercasing everything, removing stop words, and stemming / lemmatization

2 – Tokenization

- Splitting text into smaller units like words, characters, or sub words called tokens

3 – POS Tagging

- Assigning parts of speech to each token

4 – Parsing

- Analyzing the grammatical structure

5 – Named entity recognition

- identify entities such as people and organizations

6 – Sentiment Analysis

- Categorizing (classification) text

7 – Language Modeling

- Predicting the next word or sequence in text

Feature Extraction and Text Representation

- Corpus : Collection of text (think of it as the dataset)
- Vocabulary : All UNIQUE words in the corpus

Machines don't understand text, they only work with numbers! !

Solution: Vecotrs → Represent each word as bunch of numbers (vectors)

- Sparse vectors: One Hot Encoding

  → High dimensional, mostly zeros and no meaning in structure; NOT EFFICIENT FOR LEARNING

- Dense Vectors:

  → Represent relations (Feature extraction)

  - Bag of words: counts frequencies of word occurrence
  - TF-IDF weights words by importance across documents
  - Word Embeddings: LEARNS dense vector representation

N-Grams and Language Modeling

An N-gram is a sequence of N words (e.g. unigram, bigrams, ...) used to capture local word co-occurrences and build simple language models

- Unigram $$P(w) = \frac{C(w)}{m}$$ Where m is the corpus size

- Bigram $$P(x|y) = \frac{C(x\ y)}{C(x)}$$

- General N gram $$P(w_N|w_1^{N-1}) = \frac{C(w_1^{N-1}w_N)}{C(w_1^{N-1})}$$

- Markov Assumption: To approximate sequence probability, we assume the probability of a word depends only on the previous N - 1 words.

- Sequence Probability (Bigram Example): The entire sentence probability is calculated as $P(w_1^n) \approx \prod_{i=1}^{n} P(w_i|w_{i-1})$

- Matrices: A count matrix counts occurrences of word pairs in corpus while a Probability matrix divides each cell by its row sum to establish probabilities

Special Tokens

Started and end tokens (<s>,</s>) are added to help the model recognize sentence boundaries, enabling better text generation and evaluation.

Words not present in the training corpus can cause poor model performance (Out of Vocabulary words)

Solution: <unk> →

We replace unseen and rare words by this token, helping the model generalize to words it hasn't encountered during training

Limitations of N grams

1 - Data Sparsity

- A lot of zeros

2 - Limited Context

- restricted to N-1 words

3 - Computationally Complex

- Explodes with Vocab size

4 - Doesn't capture Semantics

- Works only based on word co occurrences not based on semantics and structure